Monte Carlo methods vs finite difference methods

Subash K C, Gustav Tegund, Zijie Wang
subash.k-c.9608@student.uu.se

Abstract

This assignment aims to make a thorough comparison between Monte-Carlo methods and finite difference for pricing European options under the Constant Elasticity of Variance (CEV) model. The report contains a short theory about three different types of numerical methods to price options - lattice methods, Monte-Carlo methods, and finite difference methods. The main results from implementations of Monte Carlo and finite difference methods are presented and compared, while lattice methods are not included beyond the brief overview. The discussion focuses on implementation aspects (difficulty of implementation), accuracy (convergence rate and efficiency in achieving accurate results), and memory requirements.

1 Introduction

The pricing of financial derivatives, such as European call options, is a central problem in quantitative finance. In the classical Black–Scholes framework, closed-form solutions are available under the assumption of constant volatility. However, empirical evidence suggests that volatility often depends on the level of the underlying asset price. To capture this effect, the Constant Elasticity of Variance (CEV) model was introduced as a generalization of Black–Scholes.

In the CEV model, the dynamics of the underlying stock price S_t are governed by the stochastic differential equation (SDE):

$$dS_t = rS_t\,dt + \sigma S_t^{\gamma}\,dW_t, \tag{1}$$

where r is the risk-free interest rate, $\sigma > 0$ is the volatility parameter, γ controls the elasticity of variance, and W_t is a standard Wiener process. When $\gamma = 1$, the model reduces to the Black–Scholes dynamics with constant volatility.

The price $v(t, s)$ of a European option under this model can also be characterized as the solution to a partial differential equation (PDE) derived via Itô's lemma and risk-neutral valuation. Specifically, $v(t, s)$ satisfies

$$\frac{\partial v}{\partial t} + rs\frac{\partial v}{\partial s} + \frac{1}{2}\sigma^2 s^{2\gamma}\frac{\partial^2 v}{\partial s^2} - rv = 0, \tag{2}$$

with terminal condition

$$v(T, s) = \Phi(s) = \max(s - K, 0), \tag{3}$$

for a European call option with strike price K and maturity T.

For the special case $\gamma = 1$, the CEV model reduces to the Black–Scholes model, and the option price has the exact solution

$$V(s, t) = sN(d_1) - Ke^{-r(T-t)}N(d_2), \tag{4}$$

with

$$d_1 = \frac{\ln\left(\frac{s}{K}\right) + \left(r + \frac{1}{2}\sigma^2\right)(T - t)}{\sigma\sqrt{T - t}}, \tag{5}$$

$$d_2 = d_1 - \sigma\sqrt{T - t}, \tag{6}$$

where $N(x)$ denotes the cumulative distribution function of the standard normal distribution:

$$N(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-z^2/2}\, dz. \tag{7}$$

This closed-form expression provides an important benchmark for validating numerical methods, as results obtained from Monte Carlo or finite difference approaches for $\gamma = 1$ can be compared directly to the exact Black–Scholes price.

The main objective of this report is to compare two numerical approaches for option pricing under the CEV model: Monte Carlo simulation and finite difference methods. Our comparison focuses on several key aspects:

- **Implementation complexity:** how difficult are they to implement.
- **Accuracy and convergence:** What is the convergence rate, how efficient is the method to obtain an accurate result?
- **Computational efficiency and memory requirements:**

2 Numerical methods-overview

2.1 Lattice methods

Lattice methods, also known as tree methods, are widely used for pricing options by discretizing the possible paths of the underlying asset price. In the binomial method, the time to maturity T is divided into M discrete steps of size $\Delta t = T/M$. At each time step t_i, the asset price S_i can move up to $S_i u$ or down to $S_i d$ with probability p and $1 - p$, respectively. The parameters u, d, and p are chosen to match the expected return and variance of the continuous model, typically assuming risk-neutral valuation.

The pricing procedure follows a backward induction algorithm:

1. Constructing the binomial price tree for the underlying asset.
2. Calculating option values at the final nodes using the payoff function, e.g., for a call option, $V = \max(S - K, 0)$.
3. Recursively computing option values at earlier nodes as the discounted expected value of future payoffs:
$$V_{i,j} = e^{-r\Delta t} \left[p\, V_{i+1,j+1} + (1-p)\, V_{i+1,j} \right], \tag{8}$$
where r is the risk-free interest rate.

The value at the root of the tree, $V_{0,0}$, gives the arbitrage-free price of the option. Lattice methods are simple to implement, flexible for various payoff types, and provide exact results in the limit as $M \to \infty$.

2.2 Monte-Carlo methods

In monte carlo method, we price the option by simulatin possible paths for the underlying asset. For each path, we compute the payoff at maturity and discount it back at the risk-free rate, and then average those values. Under the risk-neutral measure:

$$\widehat{V}_{\mathrm{MC}} = e^{-rT} \frac{1}{M} \sum_{m=1}^{M} \Phi(S_T^{(m)}), \qquad \Phi(s) = \max(s - K, 0). \tag{9}$$

To generate CEV paths we discretize the SDE

$$dS_t = rS_t\, dt + \sigma S_t^{\gamma}\, dW_t.$$

The Euler–Maruyama method is

$$S_{n+1} = S_n + rS_n \Delta t + \sigma S_n^{\gamma}\, \Delta W_n, \qquad \Delta W_n \sim \mathcal{N}(0, \Delta t). \tag{10}$$

A higher order scheme is Milstein:

$$S_{n+1} = S_n + rS_n\Delta t + \sigma S_n^{\gamma}\,\Delta W_n + \tfrac{1}{2}\sigma^2\gamma S_n^{2\gamma-1}\Big((\Delta W_n)^2 - \Delta t\Big). \tag{11}$$

The estimator is unbiased but converges slowly at $\mathcal{O}(M^{-1/2})$, antithetic variates and *common random numbers (CRN)* both reduce sampling error. In particular,Delta can be computed by bump–and–revalue,

$$\Delta \approx \frac{C(S_0 + \Delta S) - C(S_0 - \Delta S)}{2\Delta S}, \tag{12}$$

using CRN to lower the variance of the difference.

2.3 Finite difference methods

In finite difference methods we want to find $v(s, 0)$ knowing the terminal condition $v(s, T) = \Phi(s)$. To do this, we define a grid in space (S) and time to enable discretization of the problem:

$$s_j = j\Delta s \quad , \quad j = 0, ..., M \tag{13}$$

$$t_n = n\Delta t \quad , \quad n = 0, .., N, \tag{14}$$

where Δs and Δt are the step sizes in space and time respectively. We also have the boundary conditions (for a call option):

$$v(0, t) = 0 \tag{15}$$

$$v(s_{\text{max}}, t) = s_{max} - Ke^{-r(T-t)}. \tag{16}$$

The problem is then discretizised explicitly or implicitly using difference methods to approximate the derivatives of the Black-Scholes equation.

2.3.1 Explicit Discretization

In the explicit discretization we use central and forward approximations to compute the theta ($\frac{\partial v}{\partial t}$), gamma ($\frac{\partial^2 v}{\partial s^2}$), and delta term ($\frac{\partial v}{\partial s}$):

$$\frac{\partial v}{\partial t}(s_n, t_j) \approx \frac{v_{n,j+1} - v_{n,j}}{\Delta t} \tag{17}$$

$$\frac{\partial v}{\partial s^2}(s_n, t_j) \approx \frac{v_{n+1,j} - 2v_{n,j} + v_{n-1,j}}{(\Delta s)^2} \tag{18}$$

$$\frac{\partial v}{\partial s}(s_n, t_j) \approx \frac{v_{n+1,j} - v_{n-1,j}}{2\Delta s}. \tag{19}$$

We thereby get the price for an inner point in the grid from the previous time step by solving:

$$v_{n,j+1} = u_{n,j}v_{n+1,j} + d_{n,j}v_{n,j} + l_{n,j}v_{n-1,j} \quad \text{where,} \tag{20}$$

$$\begin{cases} u_{n,j} = \frac{\sigma^2 s_n^2}{2}\frac{\Delta t}{(\Delta s)^2} + rs_n\frac{\Delta t}{2\Delta s} \\ d_{n,j} = 1 - r_k\Delta t - 2\frac{\sigma^2 s_n^2}{2}\frac{\Delta t}{(\Delta s)^2} \\ l_{n,j} = \frac{\sigma^2 s_n^2}{2}\frac{\Delta t}{(\Delta s)^2} - rs_n\frac{\Delta t}{2\Delta s}. \end{cases} \tag{21}$$

This can be written as a matrix multiplication $v_{j+1} = A_j^{\text{explicit}}v_j + b_j^{\text{explicit}}$.

2.3.2 Implicit Discretization

In the implicit discretization we use central and backward approximations to compute the theta, gamma, and delta term:

$$\frac{\partial v}{\partial t}(s_n, t_{j+1}) \approx \frac{v_{n,j+1} - v_{n,j}}{\Delta t} \tag{22}$$

$$\frac{\partial v}{\partial s^2}(s_n, t_{j+1}) \approx \frac{v_{n+1,j+1} - 2v_{n,j+1} + v_{n-1,j+1}}{(\Delta s)^2} \tag{23}$$

$$\frac{\partial v}{\partial s}(s_n, t_{j+1}) \approx \frac{v_{n+1,j+1} - v_{n-1,j+1}}{2\Delta s}. \tag{24}$$

We now get the price for an inner point in the grid from the previous time step by solving:

$$\hat{u}_{n,j+1}v_{n+1,j+1} + \hat{d}_{n,j+1}v_{n,j+1} + \hat{l}_{n,j+1}v_{n-1,j+1} = v_{n,j} \quad \text{where,} \tag{25}$$

$$\begin{cases} \hat{u}_{n,j+1} = -\frac{\sigma^2 s_n^2}{2}\frac{\Delta t}{(\Delta s)^2} - r s_n \frac{\Delta t}{2\Delta s} \\ \hat{d}_{n,j+1} = 1 + r_k \Delta t + 2\frac{\sigma^2 s_n^2}{2}\frac{\Delta t}{(\Delta s)^2} \\ \hat{l}_{n,j+1} = -\frac{\sigma^2 s_n^2}{2}\frac{\Delta t}{(\Delta s)^2} + r s_n \frac{\Delta t}{2\Delta s}. \end{cases} \tag{26}$$

This is equivalent to solving a linear equation system on the form $A_{j+1}^{\text{implicit}} v_{j+1} = b_j^{\text{implicit}}$.

3 Numerical results

3.1 Monte-Carlo methods

For M simulated paths, the standard error scales as

$$s_M = \frac{\sigma}{\sqrt{M}}. \tag{27}$$

Therefore, on a log–log plot of s_M versus M we expect a straight line with slope $k = -1/2$:

$$k = \frac{\log\left(\sigma/\sqrt{cM}\right) - \log\left(\sigma/\sqrt{M}\right)}{\log(cM) - \log(M)} = -\frac{1}{2}. \tag{28}$$

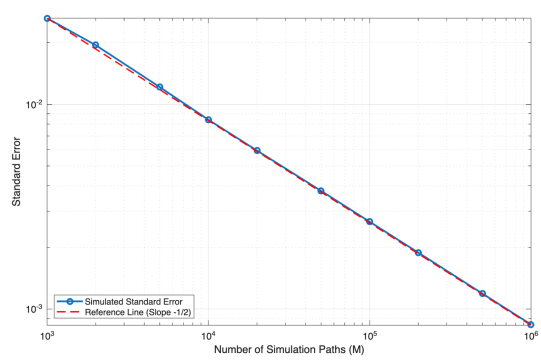
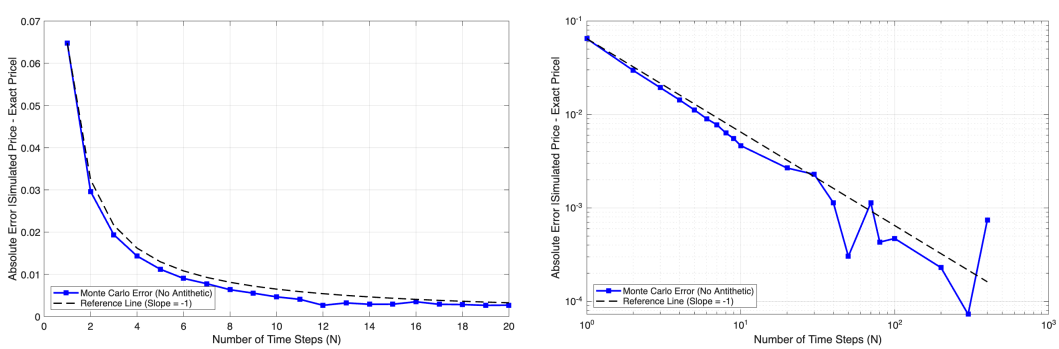


Figure 1: Sampling error vs. M

The Euler–Maruyama we use here is a first–order method to solve SDEs. When the number of paths is fixed, we expect the error to decrease and follow the reference line with slope -1 (same calculation as in the paths plot). The results follow this expectation for small and moderate N. When N increases to a certain level, the discretization error becomes very small compared with the sampling error, so the curve flattens and small fluctuations appear.

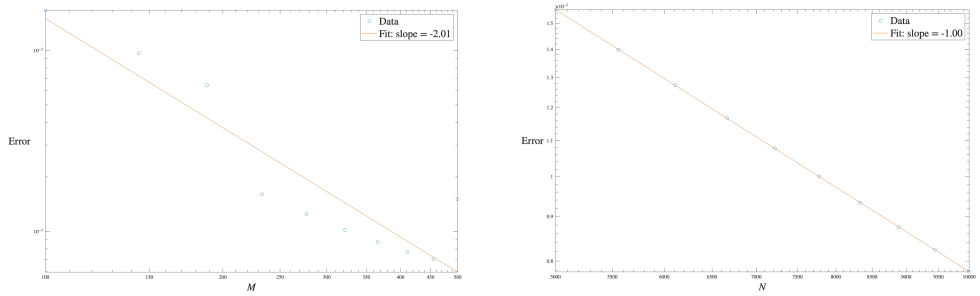


(a) Discretization error VS.N (b) Log-log plot

Figure 2: Time discretization error for Euler–Maruyama ($\gamma = 1.0$, large M). The dashed line shows the reference slope -1.

3.2 Finite difference methods

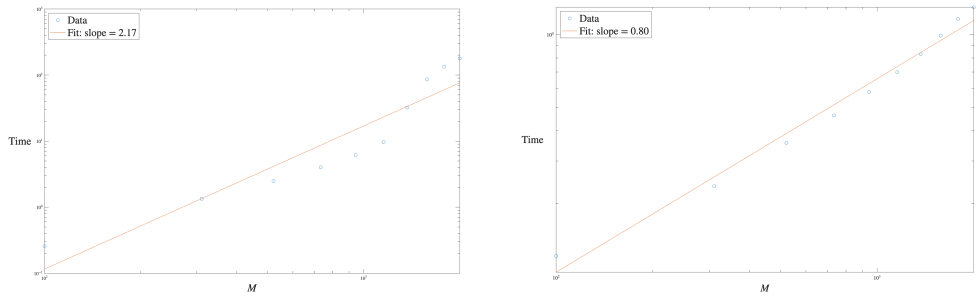
In Figure 3 we find the error convergence of the finite difference methods. Here the implicit method is used in the case of number of time steps N to keep the error in space low without running into stability issues, and the explicit method is used in the case of number of grid points in space due to faster running time.



(a) Number of grid points in space (b) Number of time steps

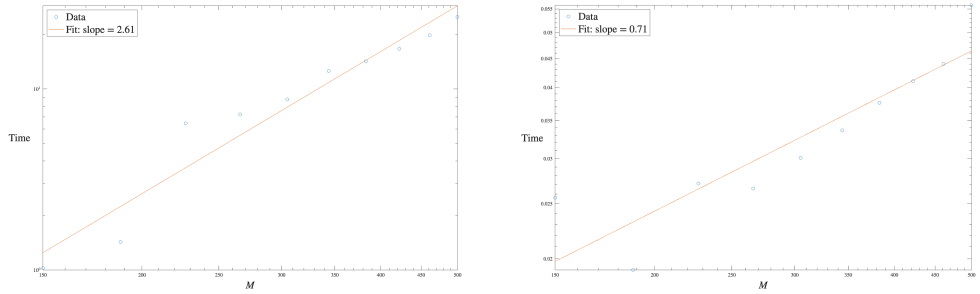
Figure 3: The error convergence of the finite difference methods analyzed against number of grid points in space in (a) and number of time steps in (b).

In Figures 4, 5 we find the computation time for the different solvers for different M values using both dense matrices and operations and sparse ones. The sparsity comes in when designing the A matrix for both solvers. Furthermore. in the case of the implicit solver the MATLAB operation `linsolve` is replaced with the backslash operator, since it is sparse-aware.



(a) Dense (b) Sparse

Figure 4: The computation time of the explicit discretization method for different M values using dense matrices and operations in (a), and sparse ones in (b).



(a) Dense (b) Sparse

Figure 5: The computation time of the implicit discretization method for different M values using dense matrices and operations in (a), and sparse ones in (b).

4 Discussion of result

4.1 Accuracy and convergence

For the Monte Carlo method we see in Figure 1 that the sample error goes down like $\mathcal{O}(M^{-1/2})$. The log-log plot shows a slope close to $-1/2$. In the time discretization test (Figure 2), the error decreases linearly with the time step, giving a slope of about -1. So both results match the expected convergence rates.
For the finite difference methods we find from the results in Figure 3 that the error decreases with a factor of one with respect to number of time steps, and a factor of two with respect to number of grid points in space. This is in aggreement with the theory since the error should follow the $\mathcal{O}(\Delta t) + \mathcal{O}(\Delta s^2)$ [3]. This is equivalent to $\mathcal{O}(N^{-1}) + \mathcal{O}(M^{-2})$, which is the same as what is found numerically in Figure 3.

4.2 Computation of the Greeks

In this section we analyze how delta can be computed using Monte Carlo methods and finite difference methods.
For the Monte Carlo method we use the bump-and-revalue with common random numbers(CRN). We run two sims at $S_0 \pm \varepsilon$, share the shocks and take a central difference. We also try antithetic (pair Z with $-Z$). Both fit the Black-Scholes Delta (0.4924), and antithetic reduces the SE by about $1.94\times$ in a similar runtime.

Table 1: MC Delta: CRN vs CRN+AV

Method	$\widehat{\Delta} \pm$ SE	Time (s)
CRN + Antithetic	0.492490 $\pm$ 2.095e$-$04	9.10
CRN	0.492350 $\pm$ 4.064e$-$04	7.47
Black-Scholes (exact)	0.4924	–

In the finite difference methods this can be done by simply looping over the number of grid points in space, and compute the delta with central difference approximations for all the inner points, a forward approximation for $s = 0$. and a backward approximation for $s = s_{\text{max}}$. This is a rather efficient operation with a complexity of $\mathcal{O}(M)$. As seen in Figure 6, the calculation is also accurate.

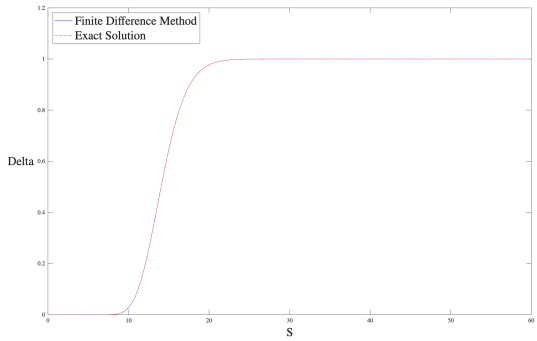


Figure 6: Calculation of the delta term with $M = 400$ and $N = 6000$ using a finite difference method compared to the exact solution.

4.3 Higher Dimensional Case

We keep (M, N) fixed and increase the number of assets d. On log-log axes, runtime grows faster than linear and bends toward a d^2 law. The fitted slope is 1.6.
The total time consists of a linear $O(Md)$ and a quadratic $O(Md^2)$ component from the correlation multiply $ZL^\top.d$ increases the quadratic part dominates, so the curve tends toward a d^2 law, while the streaming memory footprint remains $O(Md)$.

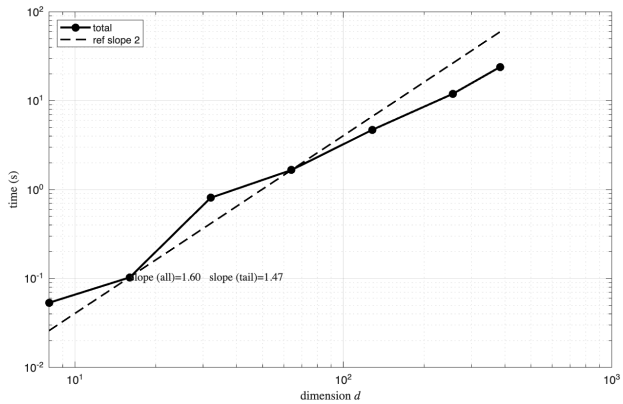


Figure 7: MC runtime vs. dimension d

In the higher dimensional case the partial differential equation becomes:

$$\frac{\partial v}{\partial t} + r\sum_{i=1}^{n}\frac{\partial v}{\partial s_i} + \frac{1}{2}\sum_{i,j=1}^{n}\sigma_i\sigma_j s_i^{\gamma} s_j^{\gamma}\frac{\partial^2 v}{\partial s_i \partial s_j} - rv = 0, \tag{29}$$

for n dimensions [4].
For the finite difference methods this means our space grid goes from a line in the one-dimensional case, to rectangular mesh in two dimensions, and finally a hypercube lattice in the n-dimensional case. This results in the number of unknowns growing exponentially with the dimensions. The matrix A becomes grows for each added dimension with another level to make it a n-level block tridiagonal matrix [5]. This matrix has to be stored in memory and if we utilize the sparse structure, the task becomes very difficult for dimensions larger than three.

4.4 Dense vs. sparse

In the finite difference methods we experimentally find the complexity of the explicit solver to be around $\mathcal{O}(M^2)$ when using dense matrices and operations, and only around $O(M)$ when using sparse matrices and operations (see Figure 4. $\mathcal{O}(M^2)$ is to be expected when for a normal matrix multiplication, while we in the sparse setting only have $3(M-1)$ nonzero elements, thus resulting in a complexity of $\mathcal{O}(M)$.

For the implicit solver we numerically find the complexity to be around $\mathcal{O}(M^{2.6})$ for in the dense setting. Here the operation of interest is the `linsolve` operation, that is based on LU-factorization. This is suggested to grow cubically [6], which is slightly faster than the experimental value. This could be explained by the matrix structure of A that is more sparse than a regular matrix, and could result in improvements in computation time. However, a big difference is seen when using sparse matrices and operations. In theory this should result in a linear complexity $O(M)$, but from Figure 5 we find it to be even lower at $O(M^{0.7})$. This could be explained by the times being relatively low and, therefor the accuracy of the complexity not being as exact. Here it was preferred to use the same M and N for both the dense and the sparse setting to see the difference in absolute time, keeping in mind that the accuracy of the complexity will be worse for the sparse case.

References

1 Ali Hirsa (2013) *Computational methods in finance.* ISBN 9781439829578..

2 Rüdiger U. Seydel (2017). *Tools for Computational Finance, 6th Edition.* Cham: Springer (Universitext). ISBN 9783319611623.

3 Elisabeth Larsson & Filip Marttala (2025). *Lecture Notes: Computational Finance: Pricing and Valuation 1TD186 12003 HT2025.* Department of Information Technology, Uppsala University, Sweden.

4 Sungkwon Kim, Daegyu Jeong, Changkyu Lee Jongho Kim (2020). "Finite Difference Method for the Multi-Asset Black–Scholes Equations." Mathematics, 8(3), 391. DOI: 10.3390/math8030391.

5 Yousef Saad (2003). *Iterative Methods for Sparse Linear Systems, Second Edition.* Philadelphia: Society for Industrial and Applied Mathematics (SIAM). ISBN 9780898715347.

6 Jesse Perla, Thomas J. Sargent John Stachurski (2025). "Numerical Linear Algebra and Factorizations," in Quantitative Economics with Julia. Available at: https://julia.quantecon.org/tools_and_techniques/numerical_linear_algebra.html

Appendix

In this appendix, we provide the Python code used to implement the numerical methods and generate the figures presented in the report. The code is also used for project-related data analysis.

Code for Monte-carlo methods

The following Python script was used to implement the Monte Carlo method and generate the corresponding figures:

```
function [option_price, std_err, S] = monte_carlo(S0, K, r, sigma,
    gamma, T, M, N, is_antithetic)
% European call option
% Inputs from assignment:
% S0 = 14;       % Initial stock price
% K = 15;        % trike price
% r = 0.1;       % Risk-free interest rate
% sigma = 0.25;  % Volatility coefficient
% T = 0.5;       % Time
% gamma = 0.8;
dt = T / N;
% Initialize a matrix to store price of all paths
S = zeros(N + 1, M);
S(1, :) = S0;
% Generate random numbers for the simulation
rng(42)
if is_antithetic
    % Antithetic Variates
    Z_half = randn(N, M / 2);
    Z = [Z_half, -Z_half];
else
    Z = randn(N, M);
end
% Step 1: Start simulation loops
for i = 1:N
    St = S(i, :);
    % Euler-Maruyama
    % dS = r*S*dt + sigma*S^gamma*dW
    % S(t+dt) = S(t) + r*S(t)*dt + sigma*S(t)^gamma*sqrt(dt)*Z
    S(i + 1, :) = St + r * St * dt + sigma * (St.^gamma) .* sqrt(dt)
    .* Z(i, :);
    % Ensure stock price does not go below zero
    S(i + 1, :) = max(S(i + 1, :), 0);
end
% Step 2: Calculate the results
S_R = S(end, :);
benefits = max(S_R - K, 0);
% Calculate the option price and standard error
option_price = exp(-r * T) * mean(benefits);
std_err = std(benefits) / sqrt(M);
end
```

Listing 1: Code for Monte-carlo method

```
clear;
clc;
close all;

S0 = 14; % Initial stock price
K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
N = 100;

```

```
% Paths
M_vec = [1e3, 2e3, 5e3, 1e4, 2e4, 5e4, 1e5, 2e5, 5e5, 1e6];

%% 2. Test Case 1: gamma = 0.8
fprintf('--- Testing gamma = 0.8 ---\n');
gamma_cev = 0.8;
std_errors = zeros(1, length(M_vec));

% Loop through each value of M
for i = 1:length(M_vec)
    M = M_vec(i);
    [option_price, std_err, ~] = monte_carlo(S0, K, r, sigma, gamma_cev, T, M, N, 0);
    std_errors(i) = std_err;
end

figure('Name', 'Standard Error vs. Paths (For Gamma=0.8 simulations)');
loglog(M_vec, std_errors, '-o', 'LineWidth', 2);
hold on;

% Ideal slope -1/2, add to see what happened
% https://www.oceanopticsbook.info/view/monte-carlo-simulation/level-2/error-estimation
% https://en.wikipedia.org/wiki/Monte_Carlo_integration
% Above links show that the standard deviation of the estimate is proportional to
% 1/sqrt(N). Hence the ideal slope of loglog is -0.5.
ref_line = std_errors(1) * (M_vec / M_vec(1)).^(-0.5);
loglog(M_vec, ref_line, 'r--', 'LineWidth', 1.5);
hold off;

xlabel('Number of Simulation Paths (M)');
ylabel('Standard Error');
legend('Simulated Standard Error', 'Reference Line (Slope -1/2)', 'Location', 'southwest');
grid on;

%% 3. Test Case 2: gamma = 1.0 with exact solutions
fprintf('--- Testing gamma = 1 ---\n');
gamma_1 = 1.0;
abs_errors = zeros(1, length(M_vec));

exact_price = bsexact(sigma, r, K, T, S0);
fprintf('Exact Black-Scholes Price = %.5f\n\n', exact_price);

% Loop through each value of M
for i = 1:length(M_vec)
    M = M_vec(i);
    rng(1);
    [option_price, ~, ~] = monte_carlo(S0, K, r, sigma, gamma_1, T, M, N, 0);
    abs_errors(i) = abs(option_price - exact_price);
end

figure('Name', 'Absolute Error vs. Paths');
loglog(M_vec, abs_errors, '-s', 'LineWidth', 2);
hold on;

% Add a reference line with slope -1/2
ref_line = abs_errors(1) * (M_vec / M_vec(1)).^(-0.5);
loglog(M_vec, ref_line, 'r--', 'LineWidth', 1.5);
hold off;

title('(\gamma = 1.0): Absolute Error vs. Number of Paths');
```

```
xlabel('Number of Simulation Paths (M)');
ylabel('Absolute Error');
legend('Simulated Absolute Error', 'Reference Line (Slope -1/2)', 'Location', 'southwest');
grid on;
```

Listing 2: Code for Monte Carlo sampling error convergence

```
clear;
clc;
close all;

S0 = 14;
K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
gamma_bs = 1.0; % Use gamma=1 as we have an exact solution for comparison

% Fix a large number of paths M to try to reduce statistical noise
M_fixed = 2e6;

N_vec = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50, 70, 80, 100, 200, 300, 400];

% --- 3. Calculate Error ---
fprintf('Number of paths (M) = %d\n\n', M_fixed);
abs_errors_N = zeros(1, length(N_vec));

% First, calculate the exact Black-Scholes price
exact_price = bsexact(sigma, r, K, T, S0);
fprintf('Exact Black-Scholes Price = %.5f\n\n', exact_price);

% lOOP
for i = 1:length(N_vec)
    N = N_vec(i);
    [option_price, ~, ~] = monte_carlo(S0, K, r, sigma, gamma_bs, T, M_fixed, N, 0);
    abs_errors_N(i) = abs(option_price - exact_price);
    fprintf('Number of Time Steps (N) = %-5d | Simulated Price = %.5f | Absolute Error = %.5f\n', N, option_price, abs_errors_N(i));
end

figure('Name', 'Discretization Error Analysis');
loglog(N_vec, abs_errors_N, 'b-s', 'LineWidth', 2, 'MarkerFaceColor', 'b', 'DisplayName', 'Monte Carlo Error (No Antithetic)');
hold on;
ref_line = abs_errors_N(1) * (N_vec / N_vec(1)).^(-1);
loglog(N_vec, ref_line, 'k--', 'LineWidth', 1.5, 'DisplayName', 'Reference Line (Slope = -1)');
hold off;
xlabel('Number of Time Steps (N)');
ylabel('Absolute Error |Simulated Price - Exact Price|');
legend('Location', 'southwest');
grid on;
set(gca, 'FontSize', 12);

%%
S0 = 14;
K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
```

```
gamma_bs = 1.0; % Use gamma=1 as we have an exact solution for comparison

% Fix a large number of paths M to reduce statistical noise and highlight discretization error
M_fixed = 2e6;

% Define the vector of time steps N to be tested
N_vec = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20];

% --- 3. Calculate Error ---
fprintf('Number of paths (M) = %d\n\n', M_fixed);
abs_errors_N = zeros(1, length(N_vec));

% First, calculate the exact Black-Scholes price
exact_price = bsexact(sigma, r, K, T, S0);
fprintf('Exact Black-Scholes Price = %.5f\n\n', exact_price);

% lOOP
for i = 1:length(N_vec)
    N = N_vec(i);
    [option_price, ~, ~] = monte_carlo(S0, K, r, sigma, gamma_bs, T, M_fixed, N, 0);
    abs_errors_N(i) = abs(option_price - exact_price);
    fprintf('Number of Time Steps (N) = %-5d | Simulated Price = %.5f | Absolute Error = %.5f\n', N, option_price, abs_errors_N(i));
end

figure('Name', 'Discretization Error Analysis');
plot(N_vec, abs_errors_N, 'b-s', 'LineWidth', 2, 'MarkerFaceColor', 'b', 'DisplayName', 'Monte Carlo Error (No Antithetic)');
hold on;
ref_line = abs_errors_N(1) * (N_vec / N_vec(1)).^(-1);
loglog(N_vec, ref_line, 'k--', 'LineWidth', 1.5, 'DisplayName', 'Reference Line (Slope = -1)');
hold off;
xlabel('Number of Time Steps (N)');
ylabel('Absolute Error |Simulated Price - Exact Price|');
legend('Location', 'southwest');
grid on;
set(gca, 'FontSize', 12);
```

Listing 3: Code for Monte Carlo time discretization error convergence

```
function out = mc_delta(S0,K,r,sigma,gamma,T,M,N,eps_rel, use_antithetic)
    if nargin < 10 || isempty(use_antithetic), use_antithetic = true; end
    if nargin < 9  || isempty(eps_rel),        eps_rel        = 1e-3; end
    if use_antithetic && mod(M,2)~=0
        error('With antithetic=true, M must be even. (M=%d)', M);
    end

    dt   = T / N;
    eps  = eps_rel * S0;
    disc = exp(-r*T);

    rng(42);
    tic;

    % --- Z     -Z ---
    if use_antithetic
        Zhalf = randn(N, M/2);
```

```
        Zreps = cat(3, Zhalf, -Zhalf);      % N x (M/2) x 2
        nrep  = 2;
    else
        Zreps = randn(N, M);                % N x M
        nrep  = 1;
    end

    Splus  = (S0 + eps) * ones(1, size(Zreps,2));
    Sminus = (S0 - eps) * ones(1, size(Zreps,2));
    sqdt = sqrt(dt);

    pay_p = zeros(size(Zreps,2), nrep);
    pay_m = zeros(size(Zreps,2), nrep);

    for rep = 1:nrep
        S_p = Splus;  S_m = Sminus;
        Z = Zreps(:,:,rep);
        for i=1:N
            Zi = Z(i,:);
            S_p = S_p + r*S_p*dt + sigma*(S_p.^gamma).*sqdt.*Zi;
            S_m = S_m + r*S_m*dt + sigma*(S_m.^gamma).*sqdt.*Zi;
            S_p = max(S_p,0); S_m = max(S_m,0);
        end
        pay_p(:,rep) = disc * max(S_p - K, 0);
        pay_m(:,rep) = disc * max(S_m - K, 0);
    end

    if use_antithetic
        %                 SE
        pay_p_avg = mean(pay_p, 2);                 % (M/2) x 1
        pay_m_avg = mean(pay_m, 2);
        Y = pay_p_avg - pay_m_avg;                  % (M/2)
        delta_samples = Y / (2*eps);
        delta_hat = mean(delta_samples);
        delta_se  = std(delta_samples) / sqrt(size(delta_samples,1));
        price_p_hat = mean(pay_p_avg);
        price_m_hat = mean(pay_m_avg);
        M_eff = M/2;
    else
        delta_samples = (pay_p - pay_m) / (2*eps);  % M x 1
        delta_hat = mean(delta_samples);
        delta_se  = std(delta_samples) / sqrt(numel(delta_samples));
        price_p_hat = mean(pay_p);
        price_m_hat = mean(pay_m);
        M_eff = M;
    end

    t_wall = toc;

    % ----        ----
    out = struct();
    out.delta    = delta_hat;
    out.se       = delta_se;
    out.eps      = eps;
    out.price_p  = price_p_hat;      %
    out.price_m  = price_m_hat;
    out.M        = M;
    out.M_eff    = M_eff;            %
    out.N        = N;
    out.antithetic = logical(use_antithetic);
    out.time     = t_wall;           %
end
```

Listing 4: Code for Monte Carlo computation of Delta

```
clear;
clc;
close all;

S0=14; K=15; r=0.1; sigma=0.25; T=0.5; gamma=1.0;
M=2e6; N=200; eps_rel=1e-2; runs=3;

% --- CRN+Antithetic ---
tA=zeros(runs,1);
resA = mc_delta(S0,K,r,sigma,gamma,T,M,N,eps_rel,true);
Delta_A = resA.delta; SE_A = resA.se; tA(1)=resA.time;
for k=2:runs, tA(k)=mc_delta(S0,K,r,sigma,gamma,T,M,N,eps_rel,true).time; end
time_med_A = median(tA);

% --- CRN (no antithetic) ---
tP=zeros(runs,1);
resP = mc_delta(S0,K,r,sigma,gamma,T,M,N,eps_rel,false);
Delta_P = resP.delta; SE_P = resP.se; tP(1)=resP.time;
for k=2:runs, tP(k)=mc_delta(S0,K,r,sigma,gamma,T,M,N,eps_rel,false).time; end
time_med_P = median(tP);

delta_exact = bsexact_delta(S0,K,r,sigma,T);

fprintf('MC-CRN+AV:     =%.6f     %.3e, time=%.2fs\n', Delta_A, SE_A, time_med_A);
fprintf('MC-CRN:        =%.6f     %.3e, time=%.2fs\n', Delta_P, SE_P, time_med_P);
fprintf('Exact BS    : %.6f\n', delta_exact);
fprintf('SE reduction (AV vs Plain):   %.2f\n', SE_P/SE_A);
```

Listing 5: Code for running Monte Carlo Delta

```
% mc_hd_scaling_clean.m -- total time vs dimension d (solid) + ref d^2 (dashed)

% params
S0=14; r=0.1; sigma=0.25; gamma=1.0; T=0.5; rng(42);
M=1e4; N=30; rho=0.3; d_vec=[8 16 32 64 128 256 384]; runs=3; R=8;

% measure total time (amplify d^2 by repeating correlation R times)
times=zeros(size(d_vec));
for k=1:numel(d_vec)
    d=d_vec(k); dt=T/N; sqdt=sqrt(dt);
    Sigma=(1-rho)*eye(d)+rho*ones(d); L=chol(Sigma,'lower');
    t=zeros(runs,1);
    for rep=1:runs
        S=S0*ones(M,d); tic;
        for n=1:N
            Z=randn(M,d);
            Xi=Z; for rr=1:R, Xi=Xi*L.'; end
            if gamma==1.0, S=S + r*S*dt + sigma*sqdt*(S.*Xi);
            else,          S=S + r*S*dt + sigma*sqdt*((S.^gamma).*Xi);
            end
            S=max(S,0);
        end
        t(rep)=toc;
    end
    times(k)=median(t);
end

% slopes (all points and tail)
p_all=polyfit(log(d_vec),log(times),1);
idx=ceil(numel(d_vec)/2):numel(d_vec);
```

```
p_tail=polyfit(log(d_vec(idx)),log(times(idx)),1);
s_all=p_all(1); s_tail=p_tail(1);
fprintf('slope(all)=%.2f, slope(tail)=%.2f\n',s_all,s_tail);

% plot: solid total + dashed ref d^2, bold and filled markers, no title
figure;
loglog(d_vec,times,'o-','MarkerFaceColor','k','MarkerEdgeColor','k','Color','k','LineWidth',2); hold on; grid on;
xref=[min(d_vec) max(d_vec)]; yref=times(idx(1))*(xref/d_vec(idx(1)))^2;
loglog(xref,yref,'k--','LineWidth',1.6);
xlabel('dimension $d$','Interpreter','latex'); ylabel('time (s)','Interpreter','latex');
legend({'total','ref slope 2'},'Location','northwest');

% inline slope text
txt=sprintf('slope (all)=%.2f   slope (tail)=%.2f',s_all,s_tail);
text(d_vec(2), times(2), txt, 'FontName','Times','FontSize',11);

% save
saveas(gcf,'mc_hd_total_vs_d.png');
```

Listing 6: Code for Monte Carlo high dimension runtime assessment

Code for finite difference methods.

The following MATLAB code was used to implement finite difference methods and generate the corresponding figures:

```
clear all
set(groot, 'defaultAxesTickLabelInterpreter','latex'); set(groot, 'defaultLegendInterpreter','latex');

K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
gamma = 1;
Smax = 4*K;
M = floor(linspace(100,500,10));
N = 10000;

dT = T/N;
dS = Smax./M;

t = linspace(0, T, N+1);

error_explicit = zeros(10,1);
for i = 1:10
    i
    S = linspace(0, Smax, M(i)+1);
    payoff = @(S) max(S-K, 0);
    V = payoff(S);

    V_explicit = sparse_explicit(sigma,S,M(i),N,V,r,Smax,K,T,t,gamma);
    exact = bsexact(sigma,r,K,T,S);

    error_explicit(i) = norm(V_explicit-exact);
end
p_expl = polyfit(log(M), log(error_explicit),1);
y_expl = exp(polyval(p_expl,log(M)));

figure(1)
plot(M,error_explicit,'o','DisplayName','Data')
```

```
set(gca, 'XScale','log', 'YScale','log');
hold on
plot(M,y_expl, 'DisplayName',sprintf('Fit: slope = %.2f',p_expl(1)))
hold off
xlabel('$M$','interpreter', 'latex','FontSize',20);
ylabel('Error','interpreter', 'latex','Rotation',0,'FontSize',20);
legend('Location','northeast','Interpreter','latex','FontSize',20);
```

Listing 7: Code for the error convergence with respect to number of grid points in space

```
clear all
set(groot, 'defaultAxesTickLabelInterpreter','latex'); set(groot, '
    defaultLegendInterpreter','latex');

K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
gamma = 1;
Smax = 4*K;
M = 40000;
N = floor(linspace(5000,10000,10));

dT = T./N;
dS = Smax/M;

S = linspace(0, Smax, M+1);
payoff = @(S) max(S-K, 0);
V = payoff(S);

error_implicit = zeros(10,1);
exact = bsexact(sigma,r,K,T,S);

for i = 1:10
    i
    t = linspace(0, T, N(i)+1);
    V_implicit = sparse_implicit(sigma,S,M,N(i),V,r,Smax,K,T,t,gamma);
    error_implicit(i) = norm(V_implicit-exact);
end

p_impl = polyfit(log(N), log(error_implicit),1);
y_impl = exp(polyval(p_impl,log(N)));

figure(2)
plot(N,error_implicit,'o','DisplayName','Data')
set(gca, 'XScale','log', 'YScale','log');
hold on
plot(N,y_impl, 'DisplayName',sprintf('Fit: slope = %.2f',p_impl(1)))
hold off
xlabel('$N$','interpreter', 'latex','FontSize',20);
ylabel('Error','interpreter', 'latex','Rotation',0,'FontSize',20);
legend('Location','northeast','Interpreter','latex','FontSize',20);
```

Listing 8: Code for the error convergence with respect to number of time steps

```
clear all
set(groot, 'defaultAxesTickLabelInterpreter','latex'); set(groot, '
    defaultLegendInterpreter','latex');

K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
gamma = 1;
Smax = 4*K;
```

```
M = floor(linspace(100,2000,10));
N = 400000;

dT = T/N;
dS = Smax./M;

t = linspace(0, T, N+1);

times_explicit = zeros(10,1);
times_explicit_sparse = zeros(10,1);
for i = 1:10
    i
    S = linspace(0, Smax, M(i)+1);
    payoff = @(S) max(S-K, 0);
    V = payoff(S);
    tStart = tic;
    V_explicit = explicit(sigma,S,M(i),N,V,r,Smax,K,T,t,gamma);
    times_explicit(i) = toc(tStart);

    tStart2 = tic;
    V_explicit_sp = sparse_explicit(sigma,S,M(i),N,V,r,Smax,K,T,t,
    gamma);
    times_explicit_sparse(i) = toc(tStart2);
end
p_expl = polyfit(log(M), log(times_explicit),1);
y_expl = exp(polyval(p_expl,log(M)));

p_expl_sp = polyfit(log(M), log(times_explicit_sparse),1);
y_expl_sp = exp(polyval(p_expl_sp,log(M)));


figure(1)
plot(M,times_explicit,'o','DisplayName','Data')
set(gca, 'XScale','log', 'YScale','log');
hold on
plot(M,y_expl, 'DisplayName',sprintf('Fit: slope = %.2f',p_expl(1)))
hold off
xlabel('$M$','interpreter', 'latex');
ylabel('Time','interpreter', 'latex','Rotation',0);
legend('Location','northwest','Interpreter','latex');


figure(2)
plot(M,times_explicit_sparse,'o','DisplayName','Data')
set(gca, 'XScale','log', 'YScale','log');
hold on
plot(M,y_expl_sp, 'DisplayName',sprintf('Fit: slope = %.2f',p_expl_sp
    (1)))
hold off
xlabel('$M$','interpreter', 'latex');
ylabel('Time','interpreter', 'latex','Rotation',0);
legend('Location','northwest','Interpreter','latex');
```

Listing 9: Code for the complexity comparison for the explicit method

```
clear all
set(groot, 'defaultAxesTickLabelInterpreter','latex'); set(groot, '
    defaultLegendInterpreter','latex');

K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
gamma = 1;
Smax = 4*K;
```

```
M = floor(linspace(150,500,10));
N = 10000;

dT = T/N;
dS = Smax./M;

t = linspace(0, T, N+1);

times_implicit = zeros(10,1);
times_implicit_sparse = zeros(10,1);
for i = 1:10
    i
    S = linspace(0, Smax, M(i)+1);
    payoff = @(S) max(S-K, 0);
    V = payoff(S);

    tStart3 = tic;
    V_implicit = implicit(sigma,S,M(i),N,V,r,Smax,K,T,t,gamma);
    times_implicit(i) = toc(tStart3);

    tStart4 = tic;
    V_implicit_sp = sparse_implicit(sigma,S,M(i),N,V,r,Smax,K,T,t,
    gamma);
    times_implicit_sparse(i) = toc(tStart4);
end

p_impl = polyfit(log(M), log(times_implicit),1);
y_impl = exp(polyval(p_impl,log(M)));

p_impl_sp = polyfit(log(M), log(times_implicit_sparse),1);
y_impl_sp = exp(polyval(p_impl_sp,log(M)));

figure(3)
plot(M,times_implicit,'o','DisplayName','Data')
set(gca, 'XScale','log', 'YScale','log');
hold on
plot(M,y_impl, 'DisplayName',sprintf('Fit: slope = %.2f',p_impl(1)))
hold off
xlabel('$M$','interpreter', 'latex');
ylabel('Time','interpreter', 'latex','Rotation',0);
legend('Location','northwest','Interpreter','latex');

figure(4)
plot(M,times_implicit_sparse,'o','DisplayName','Data')
set(gca, 'XScale','log', 'YScale','log');
hold on
plot(M,y_impl_sp, 'DisplayName',sprintf('Fit: slope = %.2f',p_impl_sp
    (1)))
hold off
xlabel('$M$','interpreter', 'latex');
ylabel('Time','interpreter', 'latex','Rotation',0);
legend('Location','northwest','Interpreter','latex');
```

Listing 10: Code for the complexity comparison for the implicit method

```
clear all
set(groot, 'defaultAxesTickLabelInterpreter','latex'); set(groot, '
    defaultLegendInterpreter','latex');

K = 15;
r = 0.1;
sigma = 0.25;
T = 0.5;
gamma = 1;
Smax = 4*K;
```

```
M = 400;

N = 6000;

dT = T./N;
dS = Smax/M;

S = linspace(0, Smax, M+1);


payoff = @(S) max(S-K, 0);
V = payoff(S);



exact = bsexact(sigma, r, K, T, S);

t = linspace(0, T, N+1);
[V_explicit, delta] = explicit_delta(sigma,S,M,N,V,r,Smax,K,T,t,gamma)
    ;
error_explicit = norm(V_explicit-exact);
exact_delta = bsexact_delta(S,K,r,sigma,T);
delta;
error_delta = norm(delta-exact_delta);

% Plot the results
figure;
plot(S, delta, 'b-', 'DisplayName', 'Finite Difference Method');
hold on;
plot(S, exact_delta, 'r--', 'DisplayName', 'Exact Solution');
xlabel('S','interpreter', 'latex','FontSize',20);
ylabel('Delta','interpreter', 'latex','Rotation',0,'FontSize',20);
legend('Location','northwest','Interpreter','latex','FontSize',20);
```

Listing 11: Code for plotting delta

```
function [output, delta] = explicit_delta(sigma,S,M,N,V,r,Smax,K,T,t,
    gamma)

output = zeros(size(V));
delta = zeros(size(V));

dT = T/N;
dS = Smax/M;


alpha = (sigma^2/2)*(dT/dS^2).*S.^(2*gamma);
beta = r*(dT/(2*dS)).*S;


l = alpha-beta;
d = 1 - r*dT - 2*alpha;
u = alpha+beta;
A = diag(d(2:end-1)) + ...
    diag(u(2:end-2), 1) + ...
    diag(l(3:end-1), -1);

Vn = V(2:end-1).';



for n = N:-1:1
    b = zeros(M-1,1);
    b(1) = 0;
    b(end) = u(end-1)*(Smax-K*exp(-r*(T-t(n))));
```

```

    Vn = A*Vn + b;
end
output(2:end-1) = Vn;
output(1) = V(1);
output(end) = Smax-K*exp(-r*T);

for j = 1:M+1
    if j == 1
        delta(j) = (output(j+1)-output(j))/dS;
    elseif j == M+1
        delta(j) = (output(j) - output(j-1))/dS;
    else
        delta(j) = (output(j+1) - output(j-1))/(2*dS);
    end
end
```

Listing 12: Code for the explicit solver including the calculation of delta

```
function output = implicit(sigma,S,M,N,V,r,Smax,K,T,t,gamma)
output = zeros(size(V));

dT = T/N;
dS = Smax/M;

alpha = (sigma^2/2)*(dT/dS^2)*S.^(2*gamma);
beta = r*(dT/(2*dS))*S;

l = -alpha + beta;
d = 1 + r*dT + 2*alpha;
u = -alpha-beta;

A = diag(d(2:end-1)) + ...
    diag(u(2:end-2), 1) + ...
    diag(l(3:end-1), -1);

Vn = V(2:end-1).';
for n = N:-1:1
    b = zeros(M-1,1);
    b(1) = 0;
    b(end) = -u(end-1)*(Smax-K*exp(-r*(T-t(n))));
    right = Vn + b;
    Vn = linsolve(A,right);
end
output(2:end-1) = Vn;
output(1) = V(1);
output(end) = Smax-K*exp(-r*T);
end
```

Listing 13: Code for implicit solver

```
function sol = bsexact(sigma, r, E, T, s)

d1 = ( log(s./E) + (r + 0.5*sigma^2)*T ) ./ (sigma*sqrt(T));
d2 = d1 - sigma*sqrt(T);

F = 0.5 .* s .* (1 + erf(d1 ./ sqrt(2))) ...
    - exp(-r*T) .* E .* 0.5 .* (1 + erf(d2 ./ sqrt(2)));

sol = F;

end
```

Listing 14: Code for the exact solution of the price V

```
function delta = bsexact_delta(S,K,r,sigma,T)
d1 = (log(S./K) + (r+(sigma^2)/2)*T)./(sigma*sqrt(T));
delta = 0.5 * (1+erf(d1 ./ sqrt(2)));
end
```

Listing 15: Code for the exact solution of delta